FLEXILOANS AGENTIC HACKATHON · TOPIC 3

Bank Statement Auto-Tag & Metrics Agent

Problem & Solution — Full Briefing Document

Underwriting · Agentic AI · MSME Lending

Theme	Underwriting Automation
Stack	Python · FastAPI · Claude claude-sonnet-4-20250514 · React 18
Hackathon	FlexiLoans Agentic Systems in Lending
Scope	3-hour MVP · Synthetic data · Local deployment

PART 1

The Problem

Meet Rajesh. He has 40 loan files to process today.

Rajesh is a Credit Process Analyst at an Indian MSME lender. Smart, experienced, and good at his job. Every morning, a fresh batch of loan applications lands on his desk. Each one comes with a bank statement — six months of transactions, sometimes 150 rows, in a plain CSV file.

His job, before any credit decision can be made, is to **read every single row**. Classify it. Understand it. And then compute four numbers from scratch that the underwriter needs to approve or reject the application.

> *"I open the statement. I scroll through it. I try to figure out — is this money his salary or is he just moving money around to look good? Is this EMI payment consistent? Did he have bounces last month? I do this for every file, every day."*
>
> — Credit Process Analyst, describing their actual morning

The real cost of doing this manually.

This is not a minor inconvenience. It is a structural bottleneck embedded in every loan application processed in India today.

35 min	150	40+	0
per statement, manual	transactions to classify	files per analyst per day	automated anomaly checks

Multiply that out: 40 files × 35 minutes = over 23 hours of manual work per analyst per day. That is not a person doing their job. That is a person being a data entry machine.

And it gets worse. **Two analysts looking at the same statement will classify things differently.** One flags a suspicious transfer. The other does not see it. The credit decision changes — not because the customer is different, but because of who happened to review the file.

The fraud that hides in plain sight.

There is one specific pattern that humans miss under time pressure — and it is the most dangerous one in MSME lending.

Date	Description	Amount
8 May 2024	`NEFT DR ACCT9988776655 FUND TRANSFER`	- 50,000
10 May 2024	`NEFT CR ACCT9988776655 RETURN TRANSFER`	+ 50,000
12 May 2024	`NEFT DR ACCT9988776655 FUND TRANSFER`	- 50,000

On paper, this looks like ■**1,00,000 in monthly inflows.** The actual net inflow to this business: **zero.**

This is a circular fund transfer — money sent out to a connected party and returned within days to artificially inflate Bank Turnover and qualify for a larger loan. It is one of the most common forms of application fraud in Indian MSME lending.

A tired analyst on their 30th file of the day, scanning 150 rows — they will miss this. The account number is the only clue, buried in a description field. Today, there is no system that catches it automatically.

PART 2

The Solution

One architectural insight changes everything.

Most teams who try to automate bank statement analysis make one critical mistake: they ask the LLM to do everything — classify transactions, compute metrics, detect anomalies, write the summary — in a single large prompt.

That approach produces output that is slow, expensive, and inconsistent. The LLM hallucinates arithmetic. It misses fraud patterns that require cross-row comparison. It costs too much to run at scale.

The right approach: **use the LLM only for what LLMs are actually good at — and use deterministic code for everything else.**

LLMs read natural language and assign meaning. They do not compute averages reliably. Rules detect patterns with certainty. LLMs explain those patterns clearly. Neither works as well without the other.

A 3-stage pipeline. Each stage does exactly one job.

1 **LLM classifies every transaction**

Claude Sonnet reads the description of each row and assigns one of 8 categories: salary, business inflow, EMI payment, bounce, circular transfer, gambling/crypto, regular expense, or other. Transactions are sent in batches of 20 — 6 API calls for a 110-row statement — keeping cost and latency low. Each row receives a category, a confidence score (0–1), and a one-line reason. Rows with confidence below 60% are flagged in red for human review. The agent knows what it does not know.

2 **Deterministic engine computes the underwriting metrics**

No AI in this stage. Pure Python and pandas. Average Bank Balance (ABB), monthly Bank Turnover (BTO), bounce ratio, and FOIR (Fixed Obligation to Income Ratio) are computed from the classified output. Same input always produces the same output. This is fully auditable — a regulator can verify every number independently. Using an LLM to compute arithmetic would introduce hallucination risk and non-determinism.

3 **Rules detect fraud. LLM explains it.**
A rule-based engine scans the full classified dataset for two patterns: circular fund flows (same account identifier in both a debit and a credit within 15 days) and bounce clusters (two or more bounces in the same calendar month). Only when a rule fires does Claude get called — once — to write a plain-English explanation for the underwriter. A clean statement costs zero additional LLM calls in this stage.

What Rajesh gets — instead of a raw spreadsheet.

Before	After
Raw CSV, 150 unsorted rows	Every row colour-coded by category
No categories, no labels	Confidence score per row, red if uncertain
No metrics computed	4 metric cards — ABB, BTO, bounce ratio, FOIR
No fraud flags	Anomaly alerts with severity + plain-English explanation
35 minutes of analyst time	12 seconds, every time
Quality depends on who reviews it	Identical output regardless of analyst

Rajesh does not have to find the fraud. It is already surfaced, already explained, already severity-rated. He spends his time on judgment — not data entry.

PART 3

Technical Depth

Three decisions made deliberately.

Why Claude Sonnet over GPT-4o?

Equivalent classification accuracy on Indian MSME transaction descriptions at approximately 40% lower cost per token. More consistent JSON schema adherence on messy inputs — fewer cases where the model wraps output in markdown fences or skips rows. For a task running at 50,000 statements per day, that cost gap compounds significantly. GPT-4o would add cost with no measurable accuracy gain on this specific classification task.

Why batches of 20 rows per call?

One row per call: 150 API calls per statement, slow and expensive. All 150 at once: one call, but circular flow detection becomes unreliable across a large context and cost per call spikes. Twenty rows is the optimal balance — 6 calls per statement, each under 800 tokens, and the 15-day window for circular flow detection fits within a single batch for the vast majority of statements.

Why rules for anomaly detection, not LLM?

An LLM asked to find circular transfers will sometimes flag patterns that do not exist, and sometimes miss ones that do. Confidence varies. A rule — the same account identifier appearing in both a debit and a credit within 15 days — either fires or it does not. Zero false negatives on the pattern defined. The LLM then explains what the rule found, which is a task it handles with consistent quality. This is separation of concerns applied to AI architecture.

Cost model and latency.

Mode	Cost per statement	Daily cost (50k statements)	Latency
MVP — full LLM, sequential	~ $0.08	~ $4,000	~ 9 seconds
Optimised — concurrent calls	~ $0.08	~ $4,000	~ 2–3 seconds

Production — hybrid classifier	**~ $0.005**	**~ $250**	**~ 2 seconds**

The production hybrid path: a fine-tuned small classifier (DistilBERT or similar) handles 95% of routine transactions cheaply. Claude is called only for rows where the small model confidence falls below 70% — roughly 5% of transactions. This achieves a 16x cost reduction with negligible accuracy loss.

PART 4

Honest Gaps & What This Unlocks

What we would build with more time.

GST cross-check

Compare bank-derived monthly turnover (BTO) to GST-filed turnover for the same period. A discrepancy above 20% is a hard flag for income misrepresentation — the single most valuable signal not built in the MVP.

Concurrent batch calls

Replace sequential API calls with asyncio.gather to reduce end-to-end latency from 9 seconds to 2–3 seconds. This is a one-function change deliberately deferred from the MVP to keep the code readable for the demo.

Account Aggregator integration

Replace CSV upload with the AA consent framework, enabling the agent to pull live bank data directly from the source. This eliminates the manual upload step and removes the risk of statement manipulation before submission.

Confidence calibration

Current 0.6 confidence threshold is heuristic. With a labelled dataset of 1,000+ statements, a precision-recall curve can be computed to set the optimal threshold — minimising false negatives on bounce and circular transfer categories, where missing a flag is far more costly than over-flagging.

What this agent actually unlocks.

We did not automate Rajesh's job. We gave him a superpower.

The agent handles the part of his job that should never have required human intelligence — reading, classifying, computing, pattern-matching across 150 rows of plain text. Rajesh handles the part that will always require human intelligence — deciding whether to trust what the numbers say, calling the applicant, reading between the lines, making the judgment call.

35 minutes → 12 seconds

Inconsistent → Auditable

Fraud missed → Fraud surfaced

Data entry → Judgment

That is what this agent does. For every single file. Every single day.